SYSTEMVERILOG · HOMEWORK

Counter 256 TB

연속 3 cycle 입력 · 조합별 pass / fail 통계

tb_counter_rsm · counter_rsm (UP · DOWN · STOP · CLEAR)

environment (heap)



run_stop · mode · clear → 3 cycle 유지 → 8개 조합 판정

```
pass_cnt_f[{run_stop, mode, clear}] ++
```

정안성

2026. 11

구조

gen → drv → DUT → mon → scb

mailbox 2개 · event 1개, virtual interface로 DUT 구동 · 관측

구현 1

입력 3 cycle 유지

gen_count 1에서만 randomize, 2 · 3은 직전 값 고정

구현 2

8개 조합별 pass / fail

{run_stop, mode, clear} 3 bit를 배열 index로 사용

결과

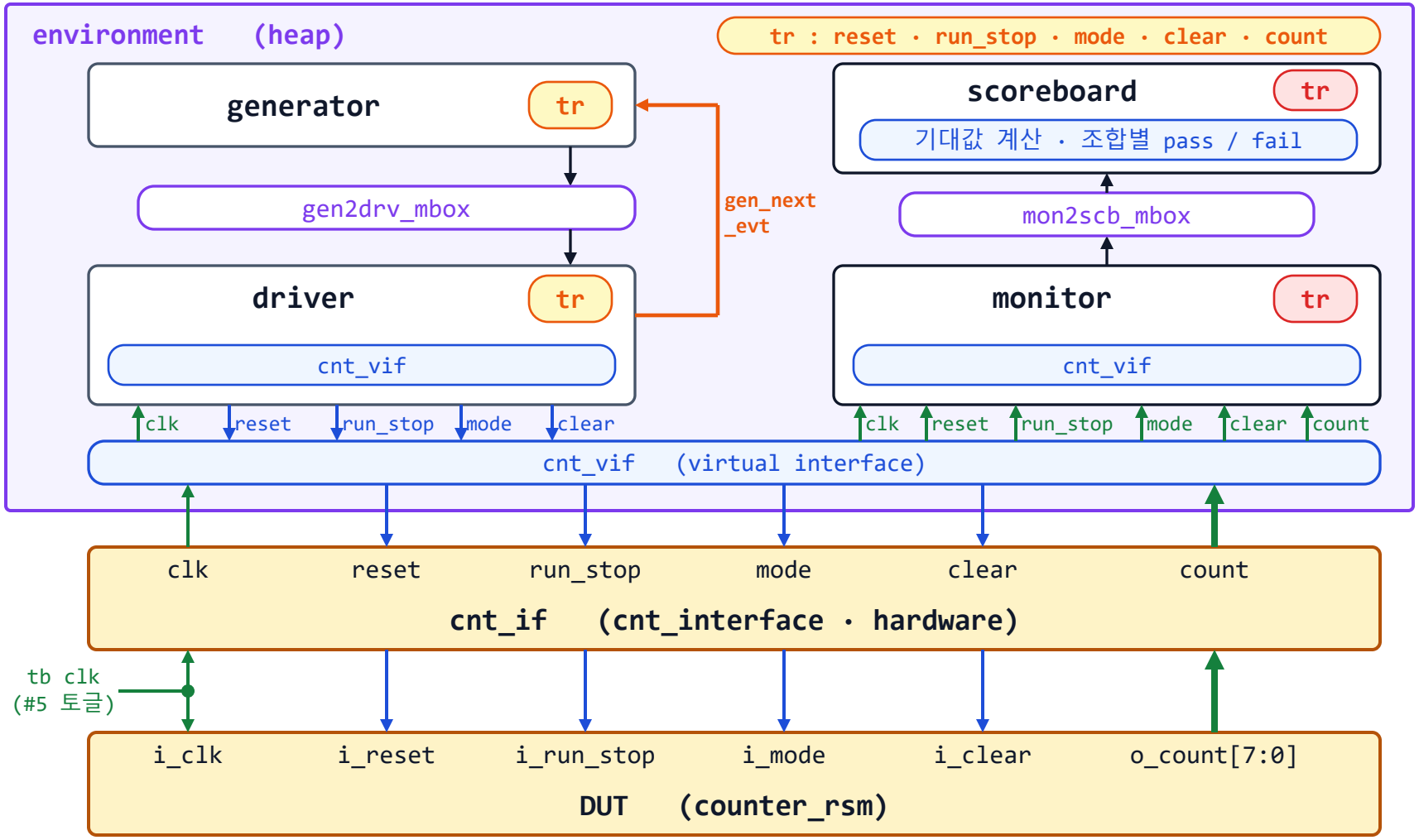
301 / 301 PASS

fail 0 · reset count 178 · 트러블 슈팅 3건 해결

검증 환경: generator → driver → DUT → monitor → scoreboard 구조

01

결론 · generator가 만든 입력을 driver가 DUT에 넣고, monitor가 읽은 출력을 scoreboard가 판정
· driver · monitor는 같은 interface(cnt_if)를 virtual interface로 공유 → driver는 입력 구동, monitor는 입력 · count 관측



generator
· 3 cycle 단위 randomize → put
· @(gen_next_evt) 대기 → 다음 tr

driver
· get → 4개 입력 구동
· @(negedge) → -> gen_next_evt

monitor
· 매 posedge + 1 ns 입력 · count 샘플
· mon2scb_mbox.put

scoreboard
· reset · clear → 0, UP +1, DOWN −1
· 조합별 pass / fail 누적 → report

environment
· preset → fork ... join_none
· 3000 ns 후 generator 정지 → report

구현 1: 3 cycle마다 한 번만 randomize, 나머지 2 cycle은 직전 값 유지

02

결론 · gen_count가 1일 때만 새 입력 조합 생성 → 같은 mode · run_stop · clear가 3 cycle 연속 유지
· randomize() with 조건으로 gen_count 2 · 3에서는 직전 값과 같도록 제한

class generator

```
bit enable ;
bit prv_reset;
bit prv_clear;
bit prv_mode;
bit prv_run_stop;

bit [1:0] cnt =1;

task run();

        while(enable)begin

            tr = new();
            tr.randomize() with{

                if(cnt>1){

                    clear == prv_clear;
                    mode  == prv_mode;
                    run_stop == prv_run_stop;
                      }
                };
            prv_clear = tr.clear;
            prv_mode  = tr.mode;
            prv_run_stop = tr.run_stop;
            gen2drv_mbox.put(tr);

    $display("+-------------------------------------------------------------------------+");
    $display(" gen_count : %5d ", cnt);
    $display("+-------------------------------------------------------------------------+");
            tr.debug_print("GEN");
            if(cnt < 3) cnt++;
            else        cnt = 1;
            @(gen_next_evt);
        end
```

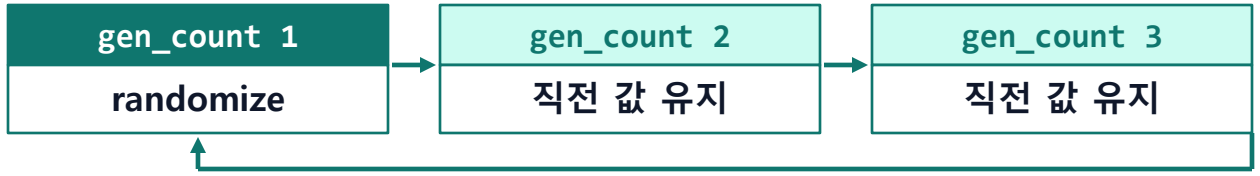
1. **prv_* · cnt (2 bit) 선언**
   직전 입력값과 cycle 순번 저장

2. **randomize() with { if (cnt > 1) … }**
   cnt 1 → 새 값, cnt 2 · 3 → 직전 값과 같도록 제한

3. **prv_*에 현재 값 저장**
   다음 cycle 제약 조건의 기준값

4. **cnt 1 → 2 → 3 → 1 순환**
   GEN 출력 뒤 증가, 3 다음 1로 초기화

gen_count 순환



· reset은 제약 대상 아님 → 매 cycle 독립 randomize (발생 확률 1 / 10)
· 다음 묶음에서 같은 값이 다시 뽑힐 경우 3 cycle 초과 유지

구현 1 결과: gen_count가 1로 돌아올 때만 입력 조합 갱신

02

결론 · 입력 조합이 3 cycle 단위로 바뀌고, 그 사이에는 같은 값 유지

· 20 – 46 ns STOP → 50 – 76 ns CLEAR, 140 – 166 ns STOP → 170 – 196 ns CLEAR 순으로 3 cycle씩 유지 확인

```
 gen_count :     1
20ns :[GEN] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
20ns :[DRV] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
26ns :[MON] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   0
26ns :[SCB] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   0
 gen_count :     2
30ns :[GEN] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
30ns :[DRV] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
36ns :[MON] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   0
36ns :[SCB] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   0
 gen_count :     3
40ns :[GEN] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
40ns :[DRV] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
46ns :[MON] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   0
46ns :[SCB] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   0
```

1 20 – 46 ns

STOP

run_stop 0
mode 1
clear 0

```
 gen_count :     1
50ns :[GEN] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
50ns :[DRV] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
56ns :[MON] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
56ns :[SCB] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
 gen_count :     2
60ns :[GEN] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
60ns :[DRV] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
66ns :[MON] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
66ns :[SCB] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
 gen_count :     3
70ns :[GEN] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
70ns :[DRV] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
76ns :[MON] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
76ns :[SCB] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
```

2 50 – 76 ns

CLEAR

run_stop 1
mode 1
clear 1

```
 gen_count :     1
140ns :[GEN] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   x
140ns :[DRV] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   x
146ns :[MON] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0
146ns :[SCB] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0
 gen_count :     2
150ns :[GEN] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   x
150ns :[DRV] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   x
156ns :[MON] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0
156ns :[SCB] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0
 gen_count :     3
160ns :[GEN] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   x
160ns :[DRV] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   x
166ns :[MON] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0
166ns :[SCB] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0
```

3 140 – 166 ns

STOP

run_stop 0
mode 0
clear 0

```
 gen_count :     1
170ns :[GEN] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   x
170ns :[DRV] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   x
176ns :[MON] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   0
176ns :[SCB] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   0
 gen_count :     2
180ns :[GEN] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   x
180ns :[DRV] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   x
186ns :[MON] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   0
186ns :[SCB] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   0
 gen_count :     3
190ns :[GEN] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   x
190ns :[DRV] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   x
196ns :[MON] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   0
196ns :[SCB] reset = 0 , run_stop = 0 ,mode = 1, clear = 1 , count=   0
```

4 170 – 196 ns

CLEAR

run_stop 0
mode 1
clear 1

구현 2: 입력 3 bit 조합을 배열 index로 사용해 조합별 pass / fail 누적

02

결론 · run_stop · mode · clear 3 bit 조합(8가지)을 그대로 배열 칸 번호로 사용
· 판정마다 해당 칸의 pass 또는 fail만 1 증가 → 조합별 통계 자동 누적

class scoreboard · 선언

```
class scoreboard;

    transaction tr;
    mailbox #(transaction) mon2scb_mbox;
    virtual cnt_interface cnt_vif;


    function new(mailbox #(transaction) mon2scb_mbox);
        this.mon2scb_mbox = mon2scb_mbox;
    endfunction

    logic [7:0] expected_count=0;
    int pass_cnt=0;
    int fail_cnt=0;
    int total_cnt =0;

    int pass_cnt_f [0:7];
    int fail_cnt_f [0:7];
    int reset_cnt ;
```

class scoreboard · 판정

```
if(expected_count == tr.count)begin
            pass_cnt ++;
            pass_cnt_f[{tr.run_stop,tr.mode,tr.clear}] ++;

            //$display("=============================================================================");
            //$display("%t  PASS : count  = %d  / expected_count = %d",$time,tr.count,expected_count);
            //$display("=============================================================================");

end

else begin
            fail_cnt ++;
            fail_cnt_f[{tr.run_stop,tr.mode,tr.clear}] ++;
            $display("=============================================================================");
            $display("%t  FAIL : count  = %d  / expected_count = %d",$time,tr.count,expected_count);
            $display("=============================================================================");
end
```

Index Table

index	run_stop	mode	clear	동작
0 (3'b000)	0	0	0	STOP
1 (3'b001)	0	0	1	CLEAR
2 (3'b010)	0	1	0	STOP
3 (3'b011)	0	1	1	CLEAR
4 (3'b100)	**1**	**0**	**0**	**UP**
5 (3'b101)	1	0	1	CLEAR
6 (3'b110)	1	1	0	DOWN
7 (3'b111)	1	1	1	CLEAR

1. **int 배열 8칸 × 2 (pass_cnt_f · fail_cnt_f)**
   칸 하나 = 32 bit int 카운터
2. **PASS → pass_cnt_f[{run_stop, mode, clear}] ++**
   예 : run_stop 1 · mode 0 · clear 0 → 칸 4 (UP)
3. **FAIL → fail_cnt_f[{run_stop, mode, clear}] ++**
   같은 index로 fail 칸 증가

구현 2 결과: 301건 전부 PASS, 8개 조합 모두 fail 0

02

결론 · **total 301 · pass 301 · fail 0, 조합별로도 fail 0**
· **report에서 for문 8회로 조합별 1줄 출력, 동작 이름은 DUT 우선순위 (CLEAR → STOP → DOWN / UP) 기준**

task report()

```
task report();

    string action;

    $display("");
    $display("+-------------------------------------------------+");
    $display("|                  TEST  REPORT                   |");
    $display("+-------------------------------------------------+");
    $display("|  total test = %5d                              |", total_cnt);
    $display("|  pass count = %5d                              |", pass_cnt);
    $display("|  fail count = %5d                              |", fail_cnt);
    $display("+-------------------------------------------------+");
    $display("| reset count = %5d                              |", reset_cnt);
    $display("+-------------------------------------------------+");
    $display("|              CASE  PASS / FAIL                  |");
    $display("+----------+------+-------+--------+-------+-------+");
    $display("| run_stop | mode | clear | action |  pass |  fail |");
    $display("+----------+------+-------+--------+-------+-------+");

    for(int i = 0; i < 8; i++)begin                                  (1)

        // 조합별 동작 이름 (모두 5글자로 맞춤)
        if(i[0])        action = "CLEAR";   // clear = 1
        else if(!i[2])  action = "STOP ";   // run_stop = 0
        else if(i[1])   action = "DOWN ";   // mode = 1
        else            action = "UP   ";   // mode = 0                (2)

        $display("|    %b     |  %b   |   %b   | %s  | %5d | %5d |",
                 i[2], i[1], i[0], action, pass_cnt_f[i], fail_cnt_f[i]);

    end

    $display("+----------+------+-------+--------+-------+-------+");
    $display("");

endtask
```

xsim 실행 결과

```
+-------------------------------------------------+
|                  TEST  REPORT                   |
+-------------------------------------------------+
|  total test =   301                             |
|  pass count =   301                             |   (3)
|  fail count =     0                             |
+-------------------------------------------------+
| reset count =   178                             |   (4)
+-------------------------------------------------+
|              CASE  PASS / FAIL                  |
+----------+------+-------+--------+-------+-------+
| run_stop | mode | clear | action |  pass |  fail |
+----------+------+-------+--------+-------+-------+
|    0     |  0   |   0   | STOP   |    24 |     0 |
|    0     |  0   |   1   | CLEAR  |    54 |     0 |
|    0     |  1   |   0   | STOP   |    37 |     0 |
|    0     |  1   |   1   | CLEAR  |    36 |     0 |   (5)
|    1     |  0   |   0   | UP     |    30 |     0 |
|    1     |  0   |   1   | CLEAR  |    48 |     0 |
|    1     |  1   |   0   | DOWN   |    42 |     0 |
|    1     |  1   |   1   | CLEAR  |    30 |     0 |
+----------+------+-------+--------+-------+-------+

$finish called at time : 3030 ns : File "C:/26_AI_CAMP/S
exit
INFO: [Common 17-206] Exiting xsim at Sat Nov  7 20:26:2
~/.../lib/sim_file$
```

(1) for (i = 0 ~ 7)
조합 8개를 1줄씩 출력

(2) action 이름 결정
clear → CLEAR, run_stop 0 → STOP, mode → DOWN / UP

(3) pass 301 / 301 · fail 0
3000 ns 동안 매 posedge 판정

(4) reset count 178
reset 또는 clear로 0이 된 판정 수

(5) 8개 조합 모두 fail 0
CLEAR 4개 조합 합 168건 (54 + 36 + 48 + 30)

03

문제 1: 0 ns에 negedge가 잡혀 preset이 10 ns 일찍 종료

결론
- 의도는 10 · 20 ns negedge 후 reset 해제, 실제는 0 · 10 ns에 끝나 10 ns에 해제
- 원인 : clk 초기값을 initial 안에서 대입 → 0 ns에 clk가 x → 0으로 바뀌며 negedge로 인식

문제



1 0 ns preset 1번째 negedge (오검출)

2 10 ns 2번째 negedge → reset 해제

`task preset()`

```
task preset();
        cnt_vif.reset = 1;
        cnt_vif.mode = 0;
        cnt_vif.clear = 0;
        cnt_vif.run_stop = 0;

        @(negedge cnt_vif.clk);   1
        $display("%t",$time);
        @(negedge cnt_vif.clk);   2
        $display("%t",$time);
        cnt_vif.reset = 0;
endtask
```

`xsim log`

```
0ns
10ns
+------------------------------
 gen_count :     1
+------------------------------
10ns :[GEN] reset = 1 , run_stop
```

0 · 10 ns에 preset 종료
→ 10 ns GEN 시작

원인

`initial begin (수정 전)`

```
initial begin

    $timeformat(-9,0,"ns",0);
    clk = 0;
    env = new(cnt_if);
    env.run();
    $finish;

end
```

clk : x → 0 ns : x → 0 = negedge

- 0 ns 대입 → 값이 x에서 0으로 변화
- x → 0도 negedge 조건에 해당
- 변화가 interface(cnt_if.clk)로 전달되는 시점
  = preset 대기 시작 이후 → 0 ns에 바로 검출

해결 1: clk를 선언과 동시에 0으로 초기화, preset 10 · 20 ns 정상 동작

03

결론 · logic clk = 0; 선언 초기화 → 0 ns에 값 변화 없음 → 첫 negedge = 10 ns
· 결과 : preset이 10 · 20 ns negedge를 기다린 뒤 20 ns에 reset 해제, generator도 20 ns 시작

해결

수정 전

```
module tb_counter_rsm();


logic clk ;

cnt_interface cnt_if(clk);
environment env;
```

수정 후

```
module tb_counter_rsm();


logic clk = 0;

cnt_interface cnt_if(clk);
environment env;
```

```
initial begin

    $timeformat(-9,0,"ns",0);
    clk = 0;
    env = new(cnt_if);
    env.run();
    $finish;

end
```

```
initial begin

    $timeformat(-9,0,"ns",0);
    env = new(cnt_if);
    env.run();
    $finish;

end
```

· 선언 초기화 = 시뮬레이션 시작 전 값 확정
· initial 안 clk = 0 삭제 → 0 ns 변화 제거

결과



1 10 ns preset 1번째 negedge

2 20 ns 2번째 → reset 해제

수정 전 log

```
0ns
10ns
+------------------------------
 gen_count :     1
+------------------------------
10ns :[GEN] reset = 1 , run_st
```

수정 후 log

```
10ns
20ns
+------------------------------
 gen_count :     1
+------------------------------
20ns :[GEN] reset = 1 , run_stop
```

문제 2: driver가 20 ns negedge를 놓쳐 첫 입력이 30 ns로 지연

03

결론 · 20 ns에 만든 입력을 driver가 30 ns에 넣음 → 25 ns monitor · scoreboard는 이전 입력으로 판정
· 원인 : preset이 끝난 20 ns edge 직후 driver 시작 → 대기 시작 이후의 edge만 감지하므로 다음 30 ns edge까지 대기

문제

xsim log

```
10ns
20ns
+-----------------------------------------------------------------------+
 gen_count :     1
+-----------------------------------------------------------------------+
20ns :[GEN] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x   1
25ns :[MON] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0   2
25ns :[SCB] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0
30ns :[DRV] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x   3
```



1 20 ns GEN　　2 25 ns MON · SCB　　3 30 ns DRV

의도 : 20 ns GEN과 함께 DRV → 25 ns MON에서 새 입력 판정

원인

class environment · run()

```
task run();  // argument 선언

    drv.preset();
    gen.enable = 1;
fork
    gen.run();
    drv.run();
    mon.run();
    scb.run();
join_none
```

class driver · run() (수정 전)

```
task run();
    forever begin
        @(negedge cnt_vif.clk);
        gen2drv_mbox.get(tr);
        tr.debug_print("DRV");

        cnt_vif.mode = tr.mode;
        cnt_vif.reset = tr.reset;
        cnt_vif.clear = tr.clear;
        cnt_vif.run_stop = tr.run_stop;
        ->gen_next_evt;



    end
```

20 ns	preset 2번째 negedge 통과 → 종료
20 ns	fork : drv.run() 시작
20 ns	@(negedge) 대기 시작 = edge 이미 지남
30 ns	다음 negedge → get · 입력 구동

해결 2: 데이터를 먼저 받아 바로 입력하고, negedge 대기는 그 뒤로 이동

03

결론 · get → 입력 구동 → @(negedge) 순서로 변경 → generator와 같은 20 ns에 입력
· 결과 : GEN · DRV 20 ns → MON · SCB 25 ns 순서로 정상 동작

해결

수정 전

```
task run();
    forever begin
        @(negedge cnt_vif.clk);
        gen2drv_mbox.get(tr);
        tr.debug_print("DRV");

        cnt_vif.mode = tr.mode;
        cnt_vif.reset = tr.reset;
        cnt_vif.clear = tr.clear;
        cnt_vif.run_stop = tr.run_stop;
        ->gen_next_evt;


    end
endtask
```

수정 후

```
task run();
    forever begin
    1 gen2drv_mbox.get(tr);
        tr.debug_print("DRV");

        cnt_vif.mode = tr.mode;
        cnt_vif.reset = tr.reset;
        cnt_vif.clear = tr.clear;
        cnt_vif.run_stop = tr.run_stop;
    2 @(negedge cnt_vif.clk);
        ->gen_next_evt;


    end
endtask
```

① get 후 바로 4개 입력 구동 ② negedge 대기 → -> gen_next_evt

· 데이터 수신 시점 = GEN 시점 → edge를 기다리지 않고 즉시 입력
· negedge 대기는 다음 transaction 요청 시점 결정용

결과

수정 전 log

```
10ns
20ns
+------------------------------------------------------------------+
 gen_count :     1
+------------------------------------------------------------------+
20ns :[GEN] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
25ns :[MON] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0
25ns :[SCB] reset = 0 , run_stop = 0 ,mode = 0, clear = 0 , count=   0
30ns :[DRV] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
```

수정 후 log

```
10ns
20ns
+------------------------------------------------------------------+
 gen_count :     1
+------------------------------------------------------------------+
20ns :[GEN] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
20ns :[DRV] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   x
25ns :[MON] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   0
25ns :[SCB] reset = 0 , run_stop = 0 ,mode = 1, clear = 0 , count=   0
+------------------------------------------------------------------+
```

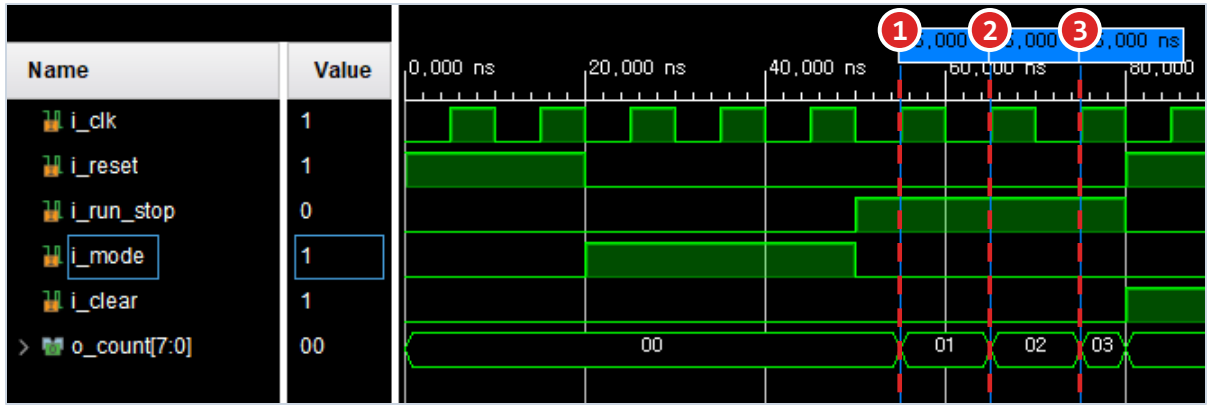
문제 3: monitor가 갱신 전 count를 읽어 카운트 cycle마다 FAIL

03

결론
- UP 구간 55 · 65 · 75 ns에서 읽은 count가 기대값보다 1 작음 → FAIL
- 원인 : DUT count는 non-blocking으로 갱신 → posedge 순간 monitor는 갱신 전 값을 읽음

문제

```
50ns :[GEN] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   x
50ns :[DRV] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   x
55ns :[MON] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   0
55ns :[SCB] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   0
================================================================================
55ns  FAIL : count  =   0  / expected_count =   1                          (1)
================================================================================
+------------------------------------------------------------------------------+
 gen_count :     2
+------------------------------------------------------------------------------+
60ns :[GEN] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   x
60ns :[DRV] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   x
65ns :[MON] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   1
65ns :[SCB] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   1
================================================================================
65ns  FAIL : count  =   1  / expected_count =   2                          (2)
================================================================================
+------------------------------------------------------------------------------+
 gen_count :     3
+------------------------------------------------------------------------------+
70ns :[GEN] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   x
70ns :[DRV] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   x
75ns :[MON] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   2
75ns :[SCB] reset = 0 , run_stop = 1 ,mode = 0, clear = 0 , count=   2
================================================================================
75ns  FAIL : count  =   2  / expected_count =   3                          (3)
================================================================================
```



1 55 ns 2 65 ns 3 75 ns

원인

class monitor · run() (수정 전)

```
task run();

        forever begin
            @(posedge cnt_vif.clk);
            tr = new();
            tr.mode = cnt_vif.mode ;
            tr.reset = cnt_vif.reset ;
            tr.clear = cnt_vif.clear ;
            tr.run_stop = cnt_vif.run_stop;
            tr.count = cnt_vif.count;
            mon2scb_mbox.put(tr);
            tr.debug_print("MON");

        end

endtask
```

posedge → monitor 깨어남 → count 읽기 (0)

NBA → DUT : count 0 → 1 갱신

55 ns 안에서 읽기가 먼저
→ 읽은 값 0, 기대값 1

해결 3: posedge 1 ns 뒤 샘플, 갱신된 count로 판정해 PASS

03

결론 · monitor에 #1 추가 → DUT 갱신이 끝난 뒤 count를 읽음
· 결과 : 56 · 66 · 76 ns 판정 모두 PASS, 전체 301건 fail 0 (p.6)

해결

수정 전

```
task run();

        forever begin
            @(posedge cnt_vif.clk);
            tr = new();
            tr.mode = cnt_vif.mode ;
            tr.reset = cnt_vif.reset ;
            tr.clear = cnt_vif.clear ;
            tr.run_stop = cnt_vif.run_stop;
            tr.count = cnt_vif.count;
            mon2scb_mbox.put(tr);
            tr.debug_print("MON");

        end

endtask
```

수정 후

```
task run();

        forever begin
            @(posedge cnt_vif.clk);
            #1;
            tr = new();
            tr.mode = cnt_vif.mode ;
            tr.reset = cnt_vif.reset ;
            tr.clear = cnt_vif.clear ;
            tr.run_stop = cnt_vif.run_stop;
            tr.count = cnt_vif.count;
            mon2scb_mbox.put(tr);
            tr.debug_print("MON");

        end

endtask
```

55 ns	posedge · DUT count 갱신 (NBA)
56 ns	+1 ns → monitor 읽기 = 갱신 후 값
56 ns	scoreboard 판정 → PASS

결과

```
+------------------------------------------------------------------------+
 gen_count :     1
+------------------------------------------------------------------------+
50ns :[GEN] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
50ns :[DRV] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
56ns :[MON] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
56ns :[SCB] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
========================================================================
56ns  PASS : count  =   0  / expected_count =   0                      (1)
========================================================================

+------------------------------------------------------------------------+
 gen_count :     2
+------------------------------------------------------------------------+
60ns :[GEN] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
60ns :[DRV] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
66ns :[MON] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
66ns :[SCB] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
========================================================================
66ns  PASS : count  =   0  / expected_count =   0                      (2)
========================================================================

+------------------------------------------------------------------------+
 gen_count :     3
+------------------------------------------------------------------------+
70ns :[GEN] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
70ns :[DRV] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   x
76ns :[MON] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
76ns :[SCB] reset = 0 , run_stop = 1 ,mode = 1, clear = 1 , count=   0
========================================================================
76ns  PASS : count  =   0  / expected_count =   0                      (3)
========================================================================
```

부록: interface · transaction 전체 코드

A

interface cnt_interface

```
interface cnt_interface(input clk);

logic reset;
logic run_stop;
logic mode;
logic clear;
logic [7:0]  count;

    property rst_zero;

        @(posedge clk) reset |=>(count == 8'd0); //reset 이 1일때, 카운트는 0이어야 한다

    endproperty


        a_rst_zero : assert property(rst_zero)
        else begin
            $display("[INTERFACE] reset = %d  counter value error !! %d ",reset,count);
        end



endinterface
```

class transaction

```
class transaction;

rand bit reset;
rand bit run_stop;
rand bit mode;
rand bit clear;
logic [7:0] count;


constraint reset_dist{
        reset dist{
            0:/ 9,
            1:/ 1
        };

}


constraint mode_dist{
        mode dist{
            0:/ 9,
            1:/ 1
        };
}

constraint run_stop_dist{
        run_stop dist{
            0:/ 9,
            1:/ 1
        };
}


constraint clear_dist{
        clear dist{
            0:/ 9,
            1:/ 1
        };
}



    function void debug_print(string name);
        $display("%t :[%s] reset = %d , run_stop = %d ,mode = %d, clear = %d , count= %d ",
                $time,name,reset,run_stop,mode,clear,count);
    endfunction


endclass
```

부록: generator · driver 전체 코드

A

class generator

```
class generator;

    transaction tr;
    mailbox #(transaction) gen2drv_mbox;
    event gen_next_evt;

    function new(mailbox #(transaction) gen2drv_mbox, event gen_next_evt);
        this.gen_next_evt =gen_next_evt;
        this.gen2drv_mbox =gen2drv_mbox;
    endfunction

    bit enable ;
    bit prv_reset;
    bit prv_clear;
    bit prv_mode;
    bit prv_run_stop;

    bit [1:0] cnt =1;

    task run();

            while(enable)begin

                tr = new();
                tr.randomize() with{

                    if(cnt>1){

                        clear == prv_clear;
                        mode  == prv_mode;
                        run_stop == prv_run_stop;
                          }
                    };
                prv_clear = tr.clear;
                prv_mode  = tr.mode;
                prv_run_stop = tr.run_stop;
                gen2drv_mbox.put(tr);

        $display("+-------------------------------------------------------------------+");
        $display(" gen_count : %5d ", cnt);
        $display("+-------------------------------------------------------------------+");
                tr.debug_print("GEN");
                if(cnt < 3) cnt++;
                else        cnt = 1;
                @(gen_next_evt);
            end

    endtask

endclass
```

class driver

```
class driver;

    transaction tr;
    mailbox #(transaction) gen2drv_mbox;

    event gen_next_evt;
    virtual cnt_interface cnt_vif;


    function new( virtual cnt_interface cnt_vif,
                                event gen_next_evt, mailbox #(transaction) gen2drv_mbox);


        this.cnt_vif = cnt_vif;
        this.gen_next_evt =gen_next_evt;
        this.gen2drv_mbox = gen2drv_mbox;

    endfunction

    task preset();
            cnt_vif.reset = 1;
            cnt_vif.mode = 0;
            cnt_vif.clear = 0;
            cnt_vif.run_stop = 0;

            @(negedge cnt_vif.clk);
            $display("%t",$time);
        @(negedge cnt_vif.clk);

            $display("%t",$time);
            cnt_vif.reset = 0;
    endtask


    task run();
        forever begin
            gen2drv_mbox.get(tr);
            tr.debug_print("DRV");

            cnt_vif.mode = tr.mode;
            cnt_vif.reset = tr.reset;
            cnt_vif.clear = tr.clear;
            cnt_vif.run_stop = tr.run_stop;
            @(negedge cnt_vif.clk);
            ->gen_next_evt;



        end
    endtask


endclass
```

부록: monitor · scoreboard 전체 코드

A

class monitor

```
class monitor;

    transaction tr;
    mailbox #(transaction) mon2scb_mbox;
    virtual cnt_interface cnt_vif;


    function new( mailbox#(transaction) mon2scb_mbox ,
                  virtual cnt_interface cnt_vif
                  );

        this.mon2scb_mbox = mon2scb_mbox;
        this.cnt_vif = cnt_vif;

    endfunction


    task run();

            forever begin
                @(posedge cnt_vif.clk);
                #1;
                tr = new();
                tr.mode = cnt_vif.mode ;
                tr.reset = cnt_vif.reset ;
                tr.clear = cnt_vif.clear ;
                tr.run_stop = cnt_vif.run_stop;
                tr.count = cnt_vif.count;
                mon2scb_mbox.put(tr);
                tr.debug_print("MON");

            end

    endtask


endclass
```

class scoreboard · run()

```
class scoreboard;

    transaction tr;
    mailbox #(transaction) mon2scb_mbox;
    virtual cnt_interface cnt_vif;


    function new(mailbox #(transaction) mon2scb_mbox);
        this.mon2scb_mbox = mon2scb_mbox;
    endfunction

    logic [7:0] expected_count=0;
    int pass_cnt=0;
    int fail_cnt=0;
    int total_cnt =0;

    int pass_cnt_f [0:7];
    int fail_cnt_f [0:7];
    int reset_cnt ;

    task run();

        forever begin

            mon2scb_mbox.get(tr);
            tr.debug_print("SCB");

            total_cnt ++;


            if(tr.reset|tr.clear) begin expected_count = 0;
                                        reset_cnt ++;
            end


            else begin

                if(tr.run_stop)begin
                    if(tr.mode) expected_count --;
                    else        expected_count ++;
                end

            end



            if(expected_count == tr.count)begin
                        pass_cnt ++;
                        pass_cnt_f[{tr.run_stop,tr.mode,tr.clear}] ++;

                        //$display("=============================================================================");
                        //$display("%t  PASS : count  = %d  / expected_count = %d",$time,tr.count,expected_count);
                        //$display("=============================================================================");

            end

            else begin
                        fail_cnt ++;
                        fail_cnt_f[{tr.run_stop,tr.mode,tr.clear}] ++;
                        $display("=============================================================================");
                        $display("%t  FAIL : count  = %d  / expected_count = %d",$time,tr.count,expected_count);
                        $display("=============================================================================");
            end

        end


    endtask
```

부록: scoreboard report · environment 전체 코드

A

class scoreboard · report()

```
task report();

    string action;

    $display("");
    $display("+-----------------------------------------------------+");
    $display("|                    TEST  REPORT                     |");
    $display("+-----------------------------------------------------+");
    $display("|  total test = %5d                                  |", total_cnt);
    $display("|  pass count = %5d                                  |", pass_cnt);
    $display("|  fail count = %5d                                  |", fail_cnt);
    $display("+-----------------------------------------------------+");
    $display("| reset count = %5d                                  |", reset_cnt);
    $display("+-----------------------------------------------------+");
    $display("|               CASE  PASS / FAIL                     |");
    $display("+----------+------+-------+--------+-------+-------+");
    $display("| run_stop | mode | clear | action |  pass |  fail |");
    $display("+----------+------+-------+--------+-------+-------+");

    for(int i = 0; i < 8; i++)begin

        // 조합별 동작 이름 (모두 5글자로 맞춤)
        if(i[0])        action = "CLEAR";   // clear = 1
        else if(!i[2])  action = "STOP ";   // run_stop = 0
        else if(i[1])   action = "DOWN ";   // mode = 1
        else            action = "UP   ";   // mode = 0

        $display("|    %b     |  %b   |   %b   | %s  | %5d | %5d |",
                i[2], i[1], i[0], action, pass_cnt_f[i], fail_cnt_f[i]);

    end

    $display("+----------+------+-------+--------+-------+-------+");
    $display("");

endtask

endclass
```

class environment

```
class environment;

    generator gen;
    driver drv;
    monitor mon;
    scoreboard scb;
    mailbox #(transaction) mon2scb_mbox;
    mailbox #(transaction) gen2drv_mbox;


    event gen_next_evt;

    function new(virtual cnt_interface cnt_vif);


        gen2drv_mbox = new();
        mon2scb_mbox = new();
        gen = new(gen2drv_mbox,gen_next_evt);
        drv = new(cnt_vif,gen_next_evt,gen2drv_mbox);
        mon = new(mon2scb_mbox,cnt_vif);
        scb = new(mon2scb_mbox);

    endfunction

    task run();  // argument 선언

            drv.preset();
            gen.enable = 1;
        fork
            gen.run();
            drv.run();
            mon.run();
            scb.run();
        join_none

            #3000;
            gen.enable = 0 ;
            #10;
            scb.report();

            disable fork;

    endtask

endclass
```